

SAFETY FILM EXAMINATIONS

SAFETY FILM



SAFETY FILM EXAMINATIONS

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

SAFETY FILM

1 A

Handle pruning.

Pruning $\rightarrow$ it is the process of removing unnecessary code.

It is the cleaning process in which helps in the optimization of the code.

B

SPD

In Syntax Directed Definition (SPD) there are 2 types of attributes.

$\rightarrow$ Synthesized

In synthesized the parent is defined by the child

$A \rightarrow BCD$

$A.s = B.s$

$A.s = C.s$

$A.s = D.s$

$\rightarrow$ Inherited.

In inherited it is either defined by a parent or a sibling

$A \rightarrow XYZ$

$Y.i = A.i$

$Y.i = X.i$

C]

Three Address Code (3AC)

3AC is a form of storing machine code which is easier for optimising and reading.

The rules are

- There should at most be 3 memory allocations.
- There should only be one operation (+, -, *, /)

D]

Strength Reduction Code Optimization.

Strength reduction code optimization is one of the many code optimization techniques.

In strength reduction code optimization, the output is not changed but the way it is approached is modified to a simpler operation to make the code fast.

before

$$x = 2^y$$

After

$$x = y \gg 1$$

E]

Basic Block.

A block consists of code which does not contain any conditional or unconditional jump within. But allowed at the first or last line.

The rules to construct a basic block is

→ Identify the header.

- ↳ The first statement is header
- ↳ The conditional/unconditional jump statement
- ↳ The following line at a conditional/unconditional jump.

→ Identify the block.

- ↳ The start of the block is a header and the end of the block is the line previous to the next header.

F]

Compiler Optimization.

The main properties of Optimization of the compiler is for

- smaller footprint
- smaller code size
- faster code
- less code space
- fewer variables.

2 A]

Syntax tree.

$a - 4 + c$

Post Fix

$a\ 4 - c +$

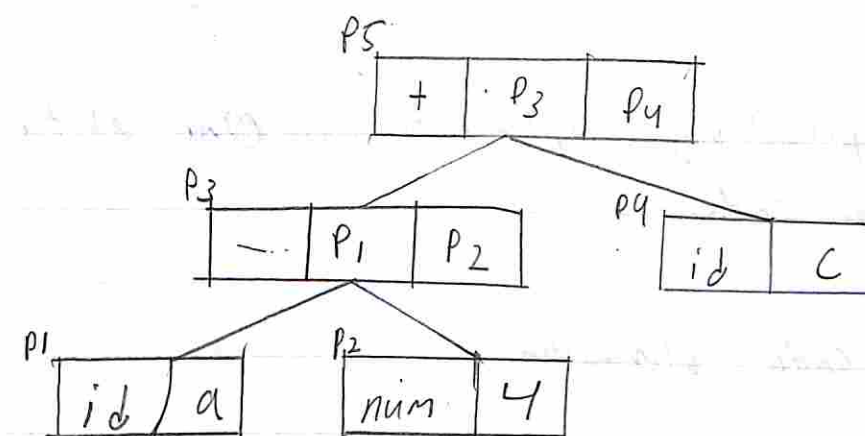
$E \rightarrow E - T$	a
$T \rightarrow T + F$	4
$E \rightarrow T$	$-$
$T \rightarrow F$	c
$F \rightarrow id$	$+$
$F \rightarrow num.$	

$P1 = \text{mkleaf}(id, a)$
 $P2 = \text{mkleaf}(num, 4)$
 $P3 = \text{mknode}(-, P1, P2)$
 $P4 = \text{mkleaf}(id, c)$
 $P5 = \text{mknode}(+, P3, P4)$

Infix

Post fix

$a - 4 + c \Rightarrow a\ 4 - c +$



Syntax tree

B] SPD

$E \rightarrow E1 + T$

$E.val = \text{mknode}(+, E1.val, T.val)$

$E \rightarrow E1 - T$

$E.val = \text{mknode}(-, E1.val, T.val)$

$E \rightarrow T$

$E.val = T.val$

$T \rightarrow (E)$

$T.val = (E.val)$

$T \rightarrow id$

$T.val = \text{mkleaf}(id, \text{entry } id)$

$T \rightarrow num.$

$T.val = \text{mkleaf}(num, \text{value})$

C

Function preserving optimization techniques

Function preserving optimization is one of the methods to optimize the code.

Function preserving optimization is one of the methods to optimize the code.

• Dead code elimination

↳ If a line of code is ~~unused~~ unused in that function then that line of code is deleted. Saving line space and memory allocation.

• Global Common Sub expression

↳ If a line of code is ~~repeated~~ repeated,

↳ If a variable declaration is repeated, then it is collapsed.

• Removal of temporary variables.

↳ If there are variables which are wasting memory allocation then it will be removed.

ex -

$a = b$
 $b = c \Rightarrow a = c.$

d

Global Common Sub expression.

In the code if any an expression is repeated

Then it means it is a waste of line space

and memory right. So if it is repeated then we

remove one and keep the other one.

$a = b + c$
 $d = a + E$
 $F = b + c \leftarrow \text{duplicate}$
 $G = F + d$

$a = b + c$
 $d = a + E$
 $G = a + d$

Optimized

3 A

$$-(a+b) * (c+d) + (a+b+c)$$

$$T1 = a+b$$

$$T2 = c+d$$

$$T1 = -T1$$

$$T3 = T1 * T2$$

$$T4 = b+c$$

$$T5 = a+T4$$

$$T6 = T3 + T5$$

Quadruple

OP	arg1	arg2	result
+	a	b	T1
+	c	d	T2
-	T1		T1
*	T1	T2	T3
+	b	c	T4
+	a	T4	T5
+	T3	T5	T6

Triple

OP	arg1	arg2
+	a	b
+	c	d
-	(0)	
*	(0)	(1)
+	b	c
+	a	(3)
+	(2)	(4)

Indirect triple
addresses.

100	(0)
101	(1)
102	(2)
103	(3)
104	(4)
105	(5)

OP	arg1	arg2
+	a	b
+	c	d
-	(0)	
*	(0)	(1)
+	b	c
+	a	(3)
+	(2)	(4)

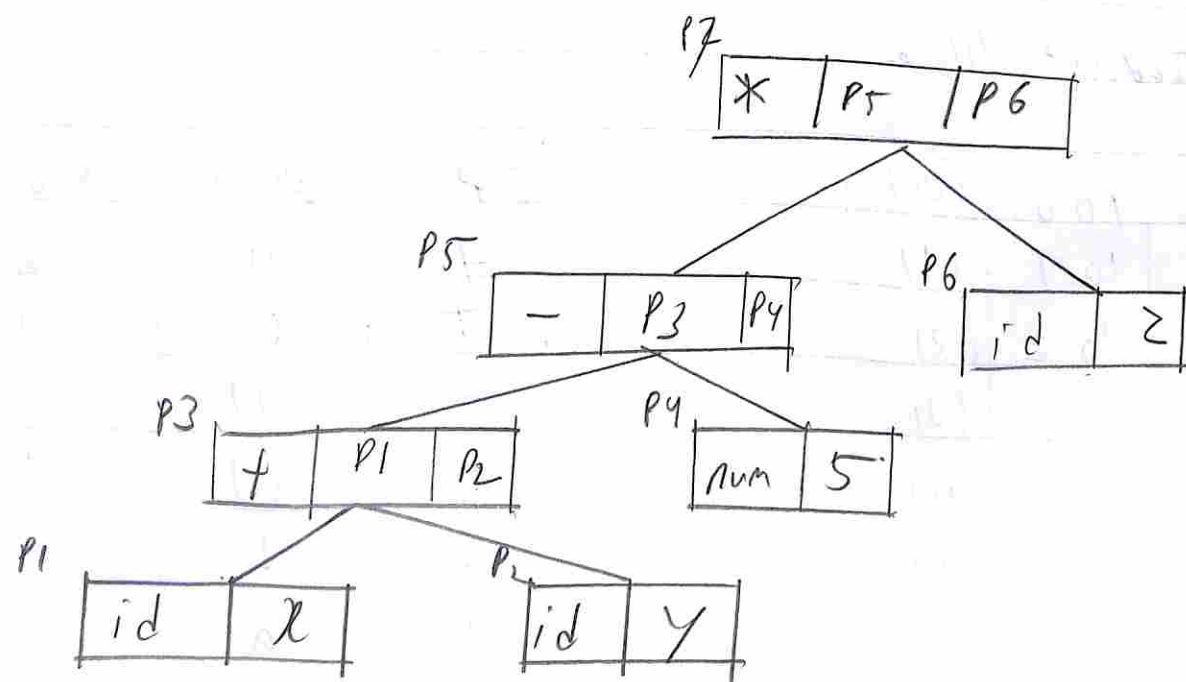
3B.

$x + y - 5 * z$

$xy + 5 - z *$

x
y
+
5
-
z
*

$P1 = \text{mkleaf}(id, x)$
 $P2 = \text{mkleaf}(id, y)$
 $P3 = \text{mknode}(+, P1, P2)$
 $P4 = \text{mkleaf}(num, 5)$
 $P5 = \text{mknode}(-, P3, P4)$
 $P6 = \text{mkleaf}(id, z)$
 $P7 = \text{mknode}(*, P5, P6)$



$E \Rightarrow E + T$

$T \rightarrow T - F$

$F \rightarrow F * G$

$F \rightarrow \text{NUM}$

$E \rightarrow id$

$T \rightarrow id$

$G \rightarrow id$

$E.val = \text{mknode}(+, E.val, T.val)$

$T.val = \text{mknode}(-, T.val, F.val)$

$F.val = \text{mknode}(*, F.val, G.val)$

$F.val = \text{mkleaf}(num, val)$

$E.val = \text{mkleaf}(id, entry)$

$T.val = \text{mkleaf}(id, entry)$

$G.val = \text{mkleaf}(id, entry)$

5A.

Dead Code ^{elimination} _{optim}

→ if any line in the code is not used then it's dead code. It only eats line space and memory but not use full. So it is

So it is removed from the code to optimize it.

Strength Reduction.

→ Each operation has a strength of its own. The operation is changed from a higher strength to lower strength so that

eg. $x = 2^y \Rightarrow x = y \gg 1$
 Code optimized

DI DAG. (3AC)

$$a = b + c$$

$$b = a - d$$

$$c = b + c$$

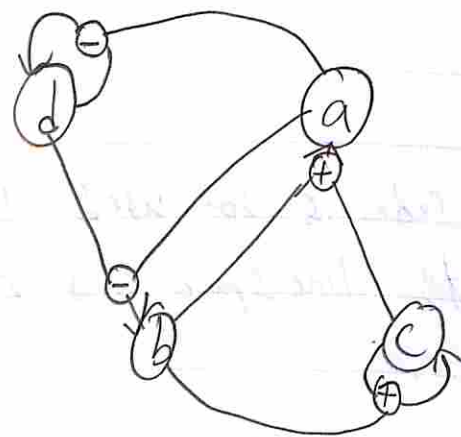
$$d = a - d$$

$$a = b + c$$

$$b = a - d$$

$$c = b + c$$

$$d = a - d$$



The rules to construct a DAG is that every node should not be repeated another time.

So if A is declared, it can't be re-declared again.

